

Experiment No.	03
Experiment Title	Regression Analysis using Linear and Regularized Models ¹
Student Name	Sharruk S
Register Number	3122247001061
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 2026

1 Objective

To implement linear and regularized regression models (Linear, Ridge, Lasso and Elastic Net Regression) for predicting a continuous target variable, evaluate their performance using multiple metrics, visualize model behaviour, and analyze overfitting, underfitting, and bias-variance characteristics.

2 Problem Statement

Given the Loan Prediction dataset containing numerical and categorical features related to loan applications, predict the continuous target variable **LoanAmount** (the sanctioned loan amount) using Linear Regression as a baseline, and Ridge, Lasso and Elastic Net Regression as regularized alternatives. The regularization strength of each regularized model is tuned using 5-fold **GridSearchCV**, and all models are compared using standard regression metrics, cross-validation performance, and diagnostic visualizations.

3 Dataset Description

Dataset Name	Loan Prediction Dataset
Dataset Source	Kaggle – Predict Loan Amount Data (repository path: <code>datasets/loan_prediction/train.csv</code>)
Original Number of Samples	614
Number of Samples Used (after dropping rows with missing target)	592
Number of Input Features	11 (after dropping <code>Loan_ID</code> and target <code>LoanAmount</code>)
Numerical Features	<code>ApplicantIncome</code> , <code>CoapplicantIncome</code> , <code>Loan_Amount_Term</code> , <code>Credit_History</code>
Categorical Features	<code>Gender</code> , <code>Married</code> , <code>Dependents</code> , <code>Education</code> , <code>Self_Employed</code> , <code>Property_Area</code> , <code>Loan_Status</code>
Missing Values (original dataset)	149 total (<code>Gender</code> : 13, <code>Married</code> : 3, <code>Dependents</code> : 15, <code>Self_Employed</code> : 32, <code>LoanAmount</code> : 22, <code>Loan_Amount_Term</code> : 14, <code>Credit_History</code> : 50)

Train-Test Split	80% Train (473 samples) – 20% Test (119 samples), <code>random_state=42</code>
------------------	--

4 Exploratory Data Analysis

A single reusable function, `create_eda_summary()`, generates a consolidated 12-subplot EDA figure covering the target variable and every input feature.

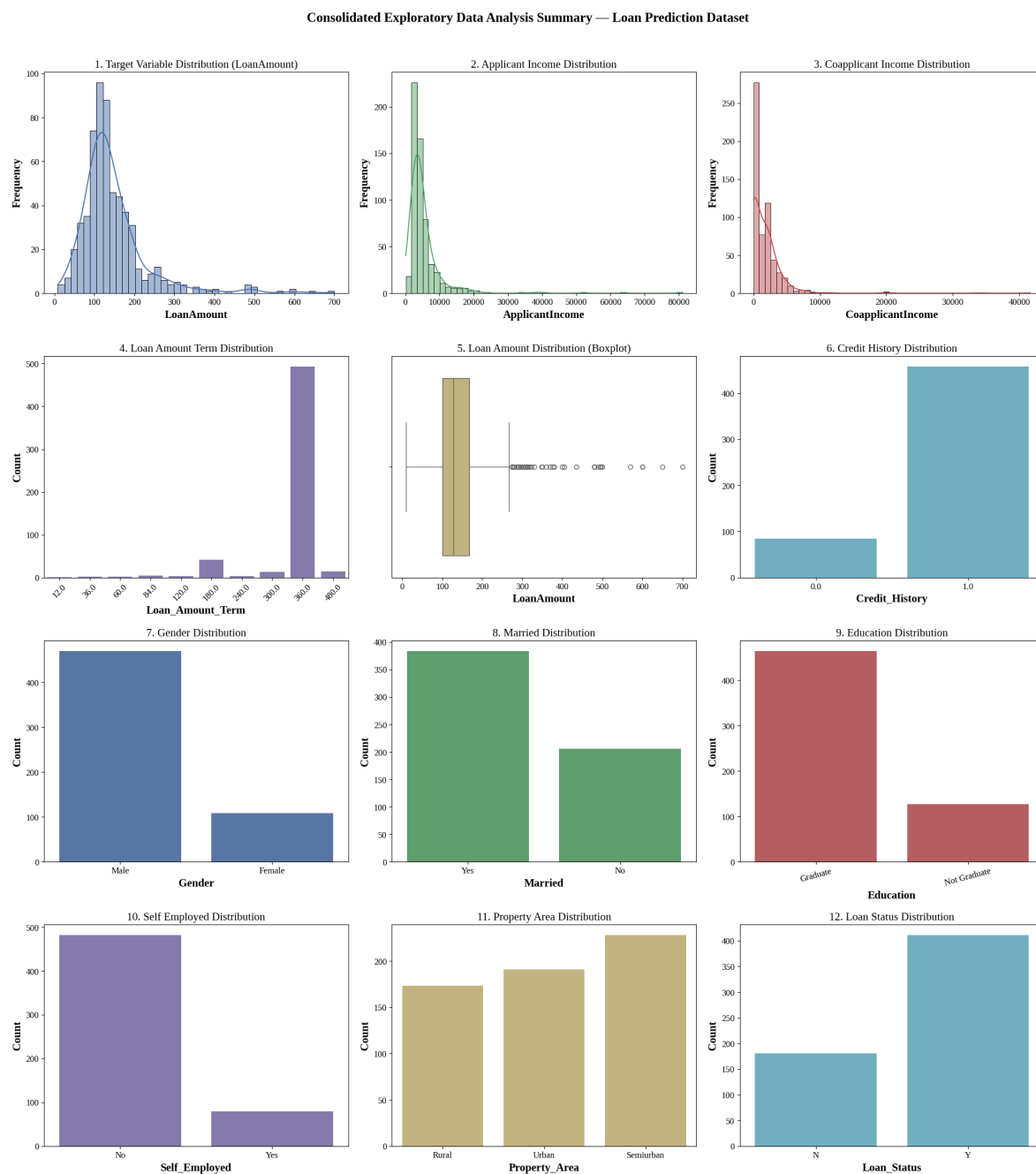


Figure 1: Consolidated 12-subplot Exploratory Data Analysis summary of the Loan Prediction dataset

Inference: The target variable `LoanAmount` (subplot 1) and `ApplicantIncome` (subplot 2) are both strongly right-skewed, with most values concentrated at the lower end and a long tail of a few very large values; the boxplot in subplot 5 confirms a substantial number of high-value outliers above the upper whisker. `Credit_History` (subplot 6) is heavily imbalanced toward the value 1 (≈ 460 of 592 records), and `Loan_Amount_Term` (subplot 4) is dominated by a single value of 360 months. Among the categorical features, `Gender` (subplot 7) and `Self_Employed` (subplot 10) are both strongly imbalanced (Male and Not-Self-Employed dominate respectively), while `Property_Area` (subplot 11) is comparatively balanced across Rural, Semiurban and Urban categories.

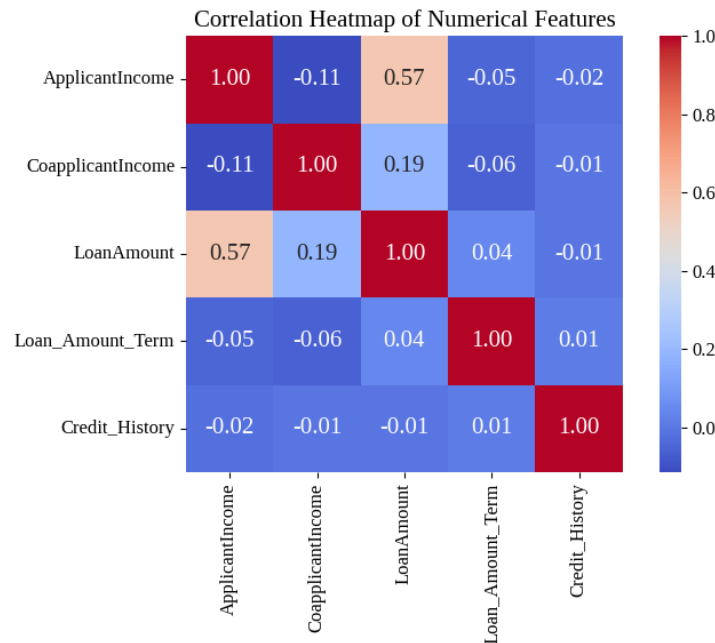


Figure 2: Correlation heatmap of the numerical features (including the target)

Inference: `ApplicantIncome` has the strongest correlation with `LoanAmount` ($r = 0.571$), confirming it as the dominant numerical predictor. `CoapplicantIncome` shows a weaker positive correlation ($r = 0.189$), while `Loan_Amount_Term` ($r = 0.039$) and `Credit_History` ($r = -0.008$) are both essentially uncorrelated with the target in this dataset.

5 Data Preprocessing

- **Missing Value Handling:** Rows with a missing target value (22 rows) were dropped, since a missing regression target cannot be validly imputed. For the remaining 149 missing values spread across the categorical and numerical predictor columns, a `scikit-learn` `ColumnTransformer` applies **median imputation** to numerical columns and **most-frequent imputation** to categorical columns.
- **Encoding:** All categorical columns (`Gender`, `Married`, `Dependents`, `Education`, `Self_Employed`, `Property_Area`, `Loan_Status`) were one-hot encoded using `OneHotEncoder(handle`
- **Feature Scaling:** All numerical columns were standardized using `StandardScaler` (zero mean, unit variance).

- **Leakage Prevention:** The entire preprocessing pipeline (imputation, encoding, scaling) is wrapped inside a `scikit-learn Pipeline` together with each regressor. This ensures imputation statistics, one-hot categories, and scaling parameters are fit **only on the training fold** at every stage – the initial train/test split, every cross-validation fold, and every `GridSearchCV` fold – so that no information from the test set (or from validation folds during tuning) leaks into training.
- **Train–Test Split:** An 80:20 split (`random_state=42`) was applied *before* any pre-processing was fit, yielding 473 training and 119 test samples.

6 Machine Learning Algorithm

6.1 Working Principle

Linear Regression fits a linear function of the input features to the target by minimizing the residual sum of squares:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n, \quad \text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Ridge Regression adds an L2 penalty ($\alpha \sum \beta_i^2$) to this loss, shrinking coefficient magnitudes without eliminating features. Lasso Regression adds an L1 penalty ($\alpha \sum |\beta_i|$), which can shrink coefficients exactly to zero, performing feature selection. Elastic Net combines both penalties, controlled by the `l1_ratio` parameter (`l1_ratio = 1` is pure Lasso, `l1_ratio = 0` is pure Ridge).

6.2 Advantages

Simple and interpretable; fast to train; regularized variants reduce overfitting risk and (for Lasso/Elastic Net) can perform automatic feature selection.

6.3 Limitations

Assumes an underlying linear relationship between features and target; sensitive to outliers; performance depends on the correct choice of regularization strength.

7 Experimental Setup

Python Version	3.12.3
pandas Version	3.0.2
NumPy Version	2.4.4
Scikit-Learn Version	1.8.0
Matplotlib Version	3.10.8
Seaborn Version	0.13.2
IDE	Jupyter Notebook

8 Hyperparameter Selection

`GridSearchCV` with 5-fold cross-validation and R^2 scoring was used to tune Ridge, Lasso and Elastic Net over the experiment-specified search spaces.

Table 2: Table 1 — Hyperparameter Tuning Summary

Model	Search Method	Best Parameters	Best CV R^2
Ridge Regression	<code>GridSearchCV</code>	$\alpha = 100$	0.2795
Lasso Regression	<code>GridSearchCV</code>	$\alpha = 10$	0.2209
Elastic Net Regression	<code>GridSearchCV</code>	$\alpha = 1$, <code>l1_ratio</code> = 0.5	0.2895

Search spaces used: Ridge $\alpha \in \{0.01, 0.1, 1, 10, 100\}$; Lasso $\alpha \in \{0.001, 0.01, 0.1, 1, 10\}$; Elastic Net $\alpha \in \{0.01, 0.1, 1, 10\}$, `l1_ratio` $\in \{0.2, 0.5, 0.8\}$.

9 Results

9.1 Cross-Validation Performance

For the final model set (baseline Linear Regression; tuned Ridge, Lasso and Elastic Net), 5-fold cross-validation was performed on the *training set*, scoring MAE, MSE and R^2 directly per fold (RMSE derived as $\sqrt{\text{MSE}}$ per fold), and averaged.

Table 3: Table 2 — Cross-Validation Performance (5-Fold Average)

Model	MAE	MSE	RMSE	R^2
Linear Regression	45.955	6926.324	79.025	0.1840
Ridge Regression	46.117	5959.862	75.779	0.2795
Lasso Regression	48.639	6480.743	78.526	0.2209
Elastic Net Regression	47.350	5800.863	75.548	0.2895

9.2 Test Set Performance

The final models were evaluated on the held-out test set (119 samples). Training time was measured directly with `time.perf_counter()` for Linear Regression, and using `GridSearchCV.refit_time()` (the time to refit the selected best estimator on the full training set) for Ridge, Lasso and Elastic Net.

Table 4: Table 3 — Test Set Performance

Model	MAE	MSE	RMSE	R^2	Training Time (s)
Linear Regression	36.988	3339.922	57.792	0.0940	0.0114
Ridge Regression	36.732	3092.854	55.613	0.1610	0.0092
Lasso Regression	37.797	3181.255	56.403	0.1370	0.0084
Elastic Net Regression	37.826	3048.685	55.215	0.1730	0.0086

Observation: Elastic Net Regression achieved the best test-set R^2 (0.1730) and joint-lowest RMSE, while Ridge Regression achieved the best cross-validated MSE/RMSE among the individually-tuned models and the lowest test MAE (36.732). Linear Regression, the unregularized baseline, performed worst on every test-set metric.

9.3 Coefficient Comparison

Coefficients were extracted directly from each fitted pipeline’s regressor step, aligned by the shared preprocessor’s output feature names. Table 5 reports the top 8 (by absolute Linear Regression coefficient) of the 15 features plotted in Figure 8.

Table 5: Table 4 — Coefficient Comparison (Top 8 of 15 Plotted Features)

Feature	Linear	Ridge	Lasso	Elastic Net
ApplicantIncome (scaled)	53.683	44.473	45.096	35.664
CoapplicantIncome (scaled)	22.064	17.442	11.914	13.200
Married_Yes	10.049	7.246	0.000	5.071
Married_No	−10.049	−7.246	−0.000	−5.071
Self_Employed_Yes	9.254	5.732	0.000	3.331
Self_Employed_No	−9.254	−5.732	−0.000	−3.331
Dependents_0	−9.228	−6.963	−0.000	−5.000
Dependents_3+	8.413	4.726	0.000	2.628

10 Result Visualization

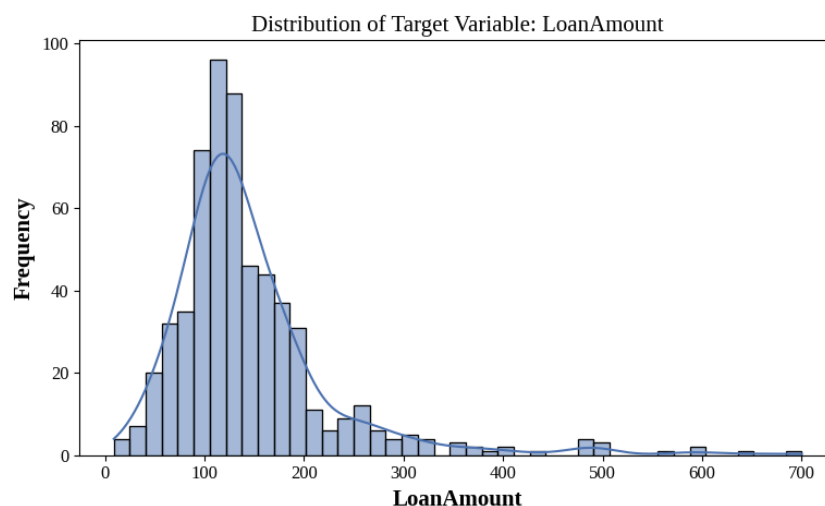


Figure 3: Distribution of the target variable, LoanAmount

Inference: The target distribution is unimodal and strongly right-skewed, with a peak near $\text{LoanAmount} \approx 110\text{--}130$ and a long tail extending past 500. This skew explains why the mean LoanAmount (146.4) is noticeably higher than the median (128), and suggests

that a small number of very large loans exert disproportionate influence on squared-error-based training objectives such as OLS/Ridge/Lasso/Elastic Net.

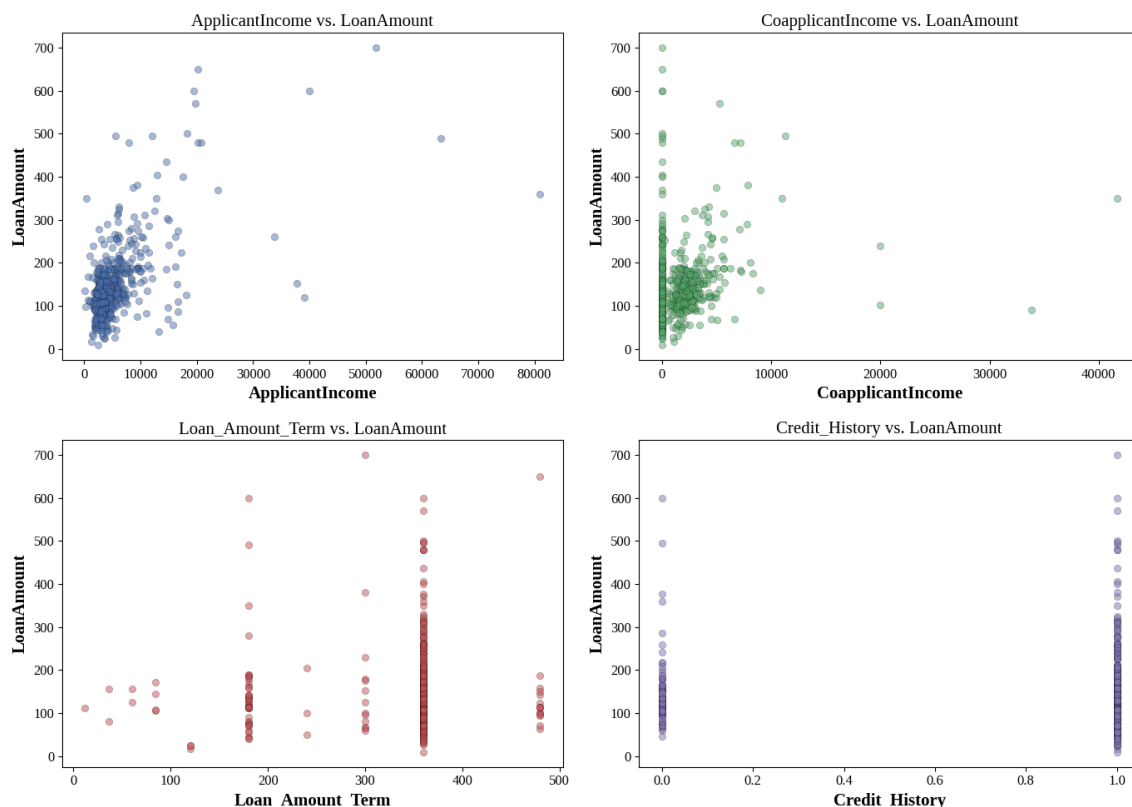


Figure 4: Feature vs. target scatter plots for the four numerical predictors

Inference: `ApplicantIncome` vs. `LoanAmount` shows the clearest positive trend among the four scatter plots, consistent with its correlation of 0.571, though the relationship is noisy and a few high-income applicants do not request correspondingly large loans. `CoapplicantIncome` shows a much weaker, more diffuse positive trend. `Loan_Amount_Term` and `Credit_History` both show no visible trend against `LoanAmount` – consistent with their near-zero correlation coefficients – and instead show vertical "stripes" of points, reflecting their small number of discrete values.

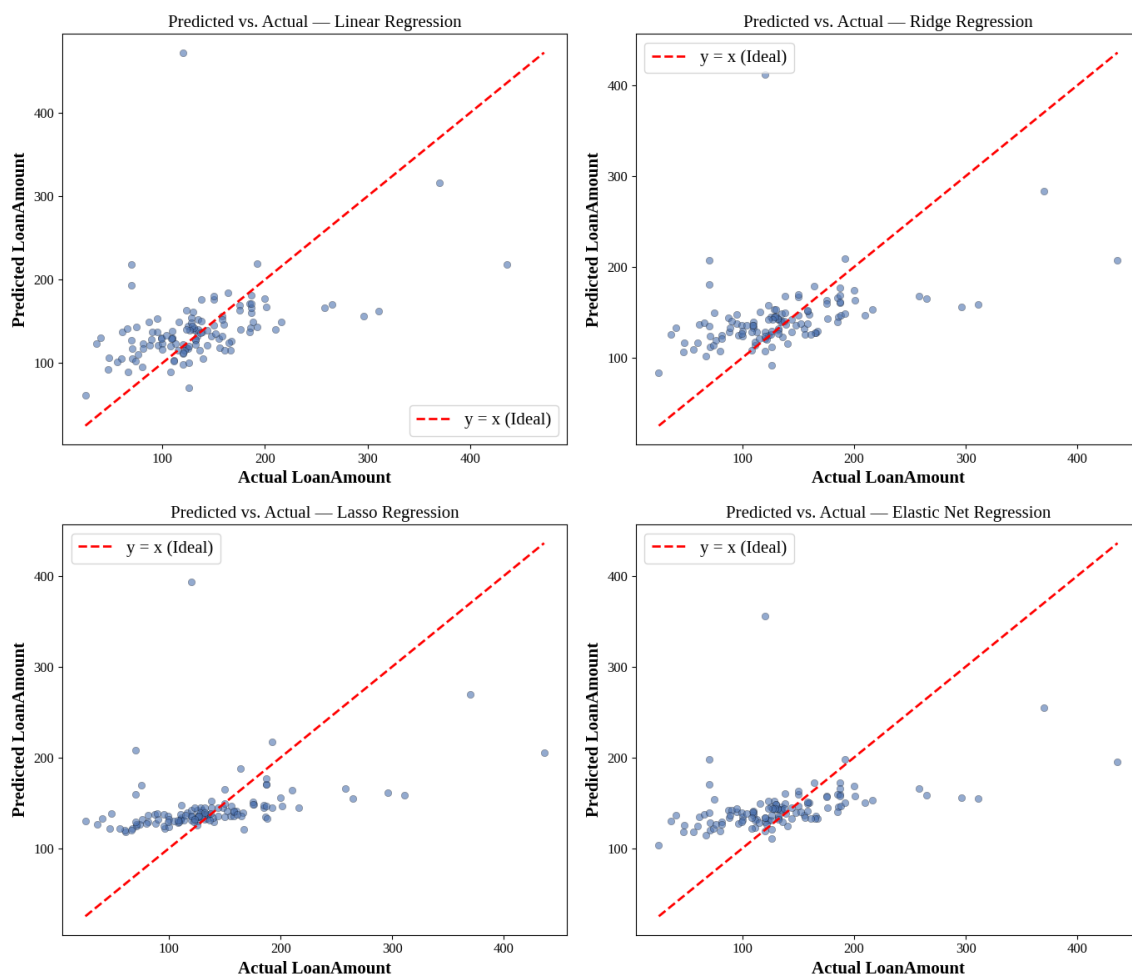


Figure 5: Predicted vs. Actual LoanAmount on the test set, for all four models

Inference: For all four models, points cluster below and to the right of the $y = x$ ideal line for large actual LoanAmount values (right side of each plot), indicating that all models systematically *under-predict* large loan amounts. For small-to-moderate actual values, points scatter more tightly around the ideal line. The overall pattern is visually similar across all four models, consistent with their similar test-set R^2 values (0.094–0.173).

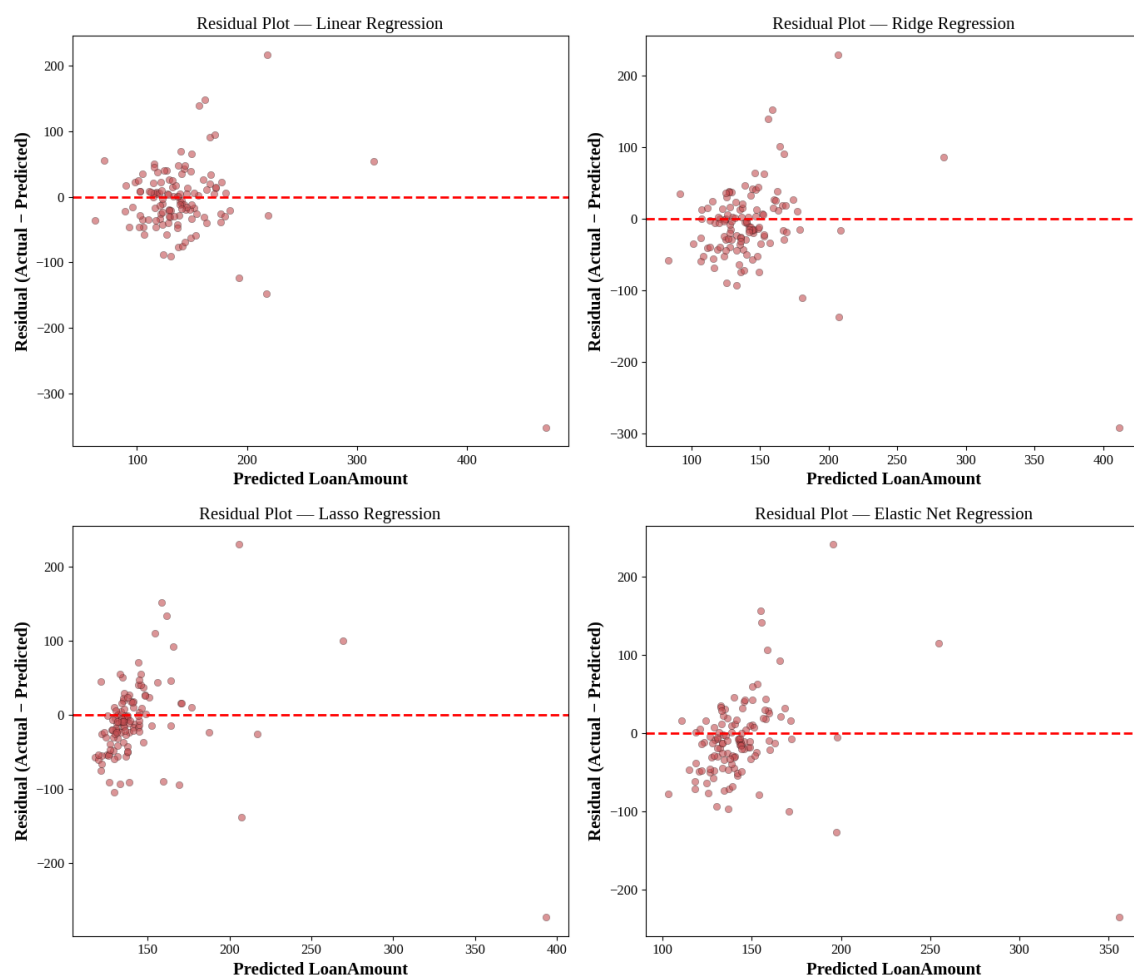


Figure 6: Residual plots (Actual — Predicted) vs. Predicted LoanAmount, for all four models

Inference: All four residual plots show residuals scattered roughly around zero for predicted values below ≈ 200 , but with a few large positive residuals (under-predictions) at moderate predicted values and one large negative outlier (an over-prediction of about -250 to -350) in every model. This single outlier and a handful of large-residual points contribute heavily to the MSE/RMSE for all models.

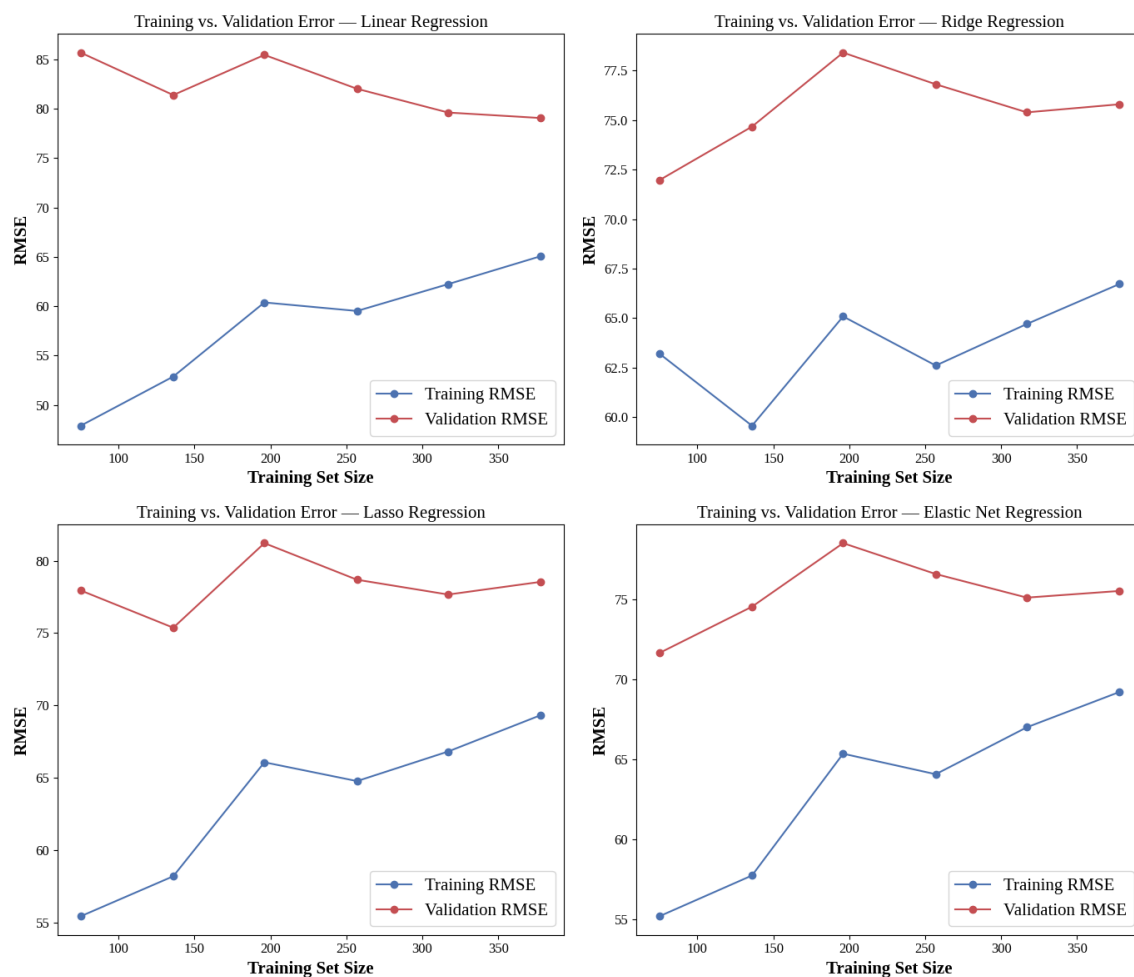


Figure 7: Training vs. Validation Error (RMSE) learning curves for all four models

Inference: For every model, the validation RMSE curve remains well above the training RMSE curve across all training set sizes, and the gap does *not* meaningfully narrow as training size increases (final gaps: Linear 13.99, Ridge 9.07, Lasso 9.22, Elastic Net 6.32). Training RMSE itself is only moderate (55–69) rather than near zero, so this is not the large-training-error/large-gap signature of overfitting; instead it is consistent with underfitting – the linear models (even after regularization) are not able to fully capture the relationship between the available features and LoanAmount.

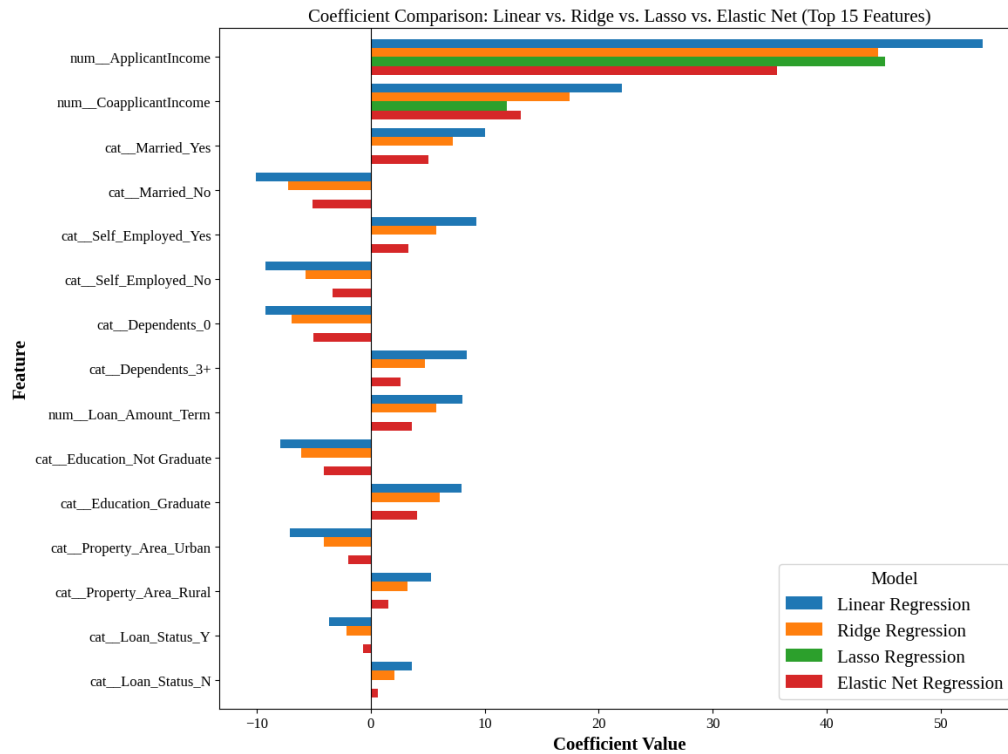


Figure 8: Coefficient comparison across Linear, Ridge, Lasso and Elastic Net (top 15 features)

Inference: `ApplicantIncome` has by far the largest-magnitude coefficient in every model, confirming it as the dominant predictor. Ridge visibly shrinks every coefficient relative to Linear Regression without zeroing any of them. Lasso (at its tuned $\alpha = 10$) is far more aggressive: of 21 total coefficients, 19 are shrunk to exactly zero, leaving only `ApplicantIncome` and `CoapplicantIncome` with non-zero weight (visible as the green bars appearing only for those two features). Elastic Net’s coefficients sit consistently between the Ridge and Lasso magnitudes, reflecting its combined L1+L2 penalty at `l1_ratio = 0.5`.

11 Discussion

11.1 Overfitting and Underfitting Analysis

Table 6: Train vs. Test R^2 (Overfitting/Underfitting Diagnostic)

Model	Train R^2	Test R^2	Gap
Linear Regression	0.4567	0.0940	0.3627
Ridge Regression	0.4394	0.1610	0.2784
Lasso Regression	0.3895	0.1370	0.2525
Elastic Net Regression	0.3929	0.1730	0.2199

Training vs. validation/test error: Every model shows a sizeable train-test R^2 gap (0.22–0.36), and the learning curves (Figure 7) confirm this: validation RMSE stays well above training RMSE at every training size without converging. However, training R^2 itself is modest for all four models (0.39–0.46) rather than close to 1.0, and training RMSE is only moderate rather than near-zero. A genuinely overfit model would show a *high* training score alongside the low test score; here the training score is itself limited. This combined evidence – a persistent train-validation gap *together with* modest absolute training performance – points to **underfitting (high bias)** as the dominant issue, rather than overfitting, for all four models on this dataset.

Effect of regularization strength: Comparing Table 2’s best CV R^2 against the baseline (untuned) linear-regression CV R^2 of 0.1840 (Table 3), all three regularized models improved cross-validated R^2 after tuning (Ridge: 0.2795, Lasso: 0.2209, Elastic Net: 0.2895). The train-test R^2 gap also shrinks monotonically from Linear Regression (0.3627) through Ridge (0.2784) and Lasso (0.2525) to Elastic Net (0.2199), showing that increasing regularization strength reduces the training/test discrepancy, even though it does not eliminate the underlying underfitting.

Does regularization improve generalization? Yes, on both the cross-validation and test-set evidence: every regularized model outperforms the unregularized Linear Regression baseline on test R^2 (Ridge 0.1610, Lasso 0.1370, Elastic Net 0.1730, all $>$ Linear’s 0.0940) and on 5-fold CV R^2 (Table 3). Elastic Net – which combines L1 and L2 penalties – gives the best overall balance of test R^2 , RMSE and train-test gap among the four models.

11.2 Bias–Variance Analysis

Linear Regression (bias behaviour): The unregularized baseline has the lowest test R^2 (0.0940) and the largest train-test gap (0.3627) of all four models. Its 5-fold CV R^2 values ranged from -0.7829 to 0.5527 (standard deviation 0.4881, the highest of all four models – see notebook Section 21 output), indicating that, in addition to its underfitting bias, the unregularized model is also highly sensitive to which rows fall into the training vs. validation fold – an indication of instability driven by the small dataset size (592 rows) combined with the influence of outlier loan amounts.

Ridge and Elastic Net (variance reduction): Ridge’s L2 penalty reduces the CV fold-to-fold R^2 standard deviation from 0.4881 (Linear) to 0.2082, and Elastic Net reduces it further still to 0.0972 – the lowest of all four models – while also achieving the highest mean CV R^2 (0.2895). This is direct evidence that the combined L1+L2 penalty in Elastic Net, and the L2 penalty in Ridge, measurably stabilize the model’s predictions across different training subsets (reduce variance) compared to the unregularized baseline, while simultaneously improving average performance – both hallmarks of a favourable bias–variance trade-off improvement from regularization on this dataset.

Lasso (feature sparsity): At its tuned $\alpha = 10$, Lasso shrinks 19 of the 21 one-hot/numeric coefficients to exactly zero, retaining only `ApplicantIncome` and `CoapplicantIncome`. This aggressive sparsity reduces Lasso’s effective model complexity substantially, but its CV R^2 standard deviation (0.2851) is higher than Ridge’s or Elastic Net’s, and its mean CV R^2 (0.2209) and test R^2 (0.1370) are both lower than Ridge’s and Elastic Net’s. This suggests that, on this dataset, Lasso’s aggressive feature elimination discards some cat-

egorical features (e.g. Married, Self_Employed, Education, Dependents) that still carry useful – if individually weak – predictive signal, so pure L1 regularization is less effective here than the combined penalty used by Elastic Net.

12 Conclusion

This experiment implemented Linear, Ridge, Lasso and Elastic Net regression models, inside leakage-safe `scikit-learn` pipelines, to predict the sanctioned loan amount from the Loan Prediction dataset. After 5-fold `GridSearchCV` tuning over the specified search spaces, Elastic Net Regression ($\alpha = 1$, `l1_ratio`= 0.5) produced the best cross-validated R^2 (0.2895) and the best test-set R^2 (0.1730) and RMSE (55.215) of the four models, while Ridge Regression ($\alpha = 100$) gave the lowest test MAE (36.732). All four models displayed clear signs of **underfitting** rather than overfitting – a persistent train-validation RMSE gap combined with only moderate absolute training performance – and regularization (most effectively the combined L1+L2 penalty in Elastic Net) measurably reduced both this gap and the fold-to-fold variance of predictions relative to the unregularized baseline. This indicates that, for further improvement on this dataset, engineering additional or higher-signal features would likely be more productive than further tuning of regularization strength alone.

13 References

1. Scikit-learn Documentation, *Linear Models*. Available: https://scikit-learn.org/stable/modules/linear_model.html
2. Scikit-learn Documentation, *Hyperparameter Optimization (GridSearchCV)*. Available: https://scikit-learn.org/stable/modules/grid_search.html
3. Scikit-learn Documentation, *Cross-validation: evaluating estimator performance*. Available: https://scikit-learn.org/stable/modules/cross_validation.html
4. Kaggle, *Predict Loan Amount / Loan Prediction Dataset*. Available: <https://www.kaggle.com/>
5. T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009.